

Module: Problem 6

This module provides two simple in-place sorting utilities that operate by repeatedly swapping out-of-order elements. Both functions print the list before sorting, after each swap, and finally after sorting completes. They modify the input list directly and do **not** return a value.

Functions

`swapSort(L)`

Description:

Implements a basic selection-sort-like algorithm. For each index `i` it scans the elements to its right (`j > i`) and swaps whenever it finds a smaller element, progressively moving the smallest remaining value toward the front of the list.

Parameters: - `L` (*list*): A mutable sequence containing comparable items (e.g., integers, strings). The function expects the items to support the `<` operator.

Returns:

`None`. The list `L` is sorted **in place**; the function's side-effects are the printed diagnostics.

Examples:

```
# Example 1 - sorting a list of strings
result = swapSort(['a', 'b', 'c']) # → None
# Console output:
# Original L:  ['a', 'b', 'c']
# Final L:    ['a', 'b', 'c']

# Example 2 - sorting a list of integers
result = swapSort([1, 2, 3])      # → None
# Console output:
# Original L:  [1, 2, 3]
# Final L:    [1, 2, 3]

# Example 3 - list with a single element
result = swapSort(['test'])      # → None
# Console output:
# Original L:  ['test']
# Final L:    ['test']
```

Edge Cases / Notes: - The function mutates `L`; callers should retain a reference to the original list if the unsorted order is needed later. - Although the docstring mentions “list of integers”, any comparable type works (e.g., strings). Passing non-comparable items will raise a `TypeError`. - No value is returned; the only observable result is the printed progress and the mutated list.

`modSwapSort(L)`

Description:

A variant of `swapSort` where the inner loop iterates over **all** indices (`j` from `0` to `len(L)-1`) instead of only the elements to the right of `i`. This leads to additional (often redundant) swaps but still results in a sorted list after completion.

Parameters: - `L` (*list*): A mutable sequence of comparable items (e.g., integers, strings). Elements must support the `<` operator.

Returns:

`None`. The list `L` is sorted **in place**, with diagnostic prints showing each swap.

Examples:

```
# Example 1 - sorting a reverse-ordered list of strings
result = modSwapSort(['c', 'b', 'a'])    # → None
# Console output (abridged):
# Original L:  ['c', 'b', 'a']
# [ ... intermediate swap prints ... ]
# Final L:  ['a', 'b', 'c']

# Example 2 - list with a single element
result = modSwapSort(['test'])           # → None
# Console output:
# Original L:  ['test']
# Final L:  ['test']

# Example 3 - sorting a reverse-ordered list of integers
result = modSwapSort([3, 2, 1])          # → None
# Console output (abridged):
# Original L:  [3, 2, 1]
# [ ... intermediate swap prints ... ]
# Final L:  [1, 2, 3]
```

Edge Cases / Notes: - Like `swapSort`, this function mutates the input list directly. - The full-range inner loop makes the algorithm less efficient ($O(n^2)$ swaps with many unnecessary operations) compared to `swapSort`. - Works with any comparable element type; non-comparable items will raise a `TypeError`. - Returns `None`; only side-effects (printing and in-place sorting) are observable.